User Interface Design

Chapter Study Notes

Software Engineering: A Practitioner's Approach
Roger S. Pressman — 7th Edition

Table of Contents

Topic 1 — Introduction to Interface Design
Topic 2 — Golden Rule 1 — Allow Users to Maintain Control
Topic 3 — Golden Rule 2 — Reduce User's Memory Load
Topic 4 — Golden Rule 3 — Make a Consistent Interface
Topic 5 — UI Design Models
Topic 6 — UI Analysis & Design Process
Topic 7 — Interface Analysis — User Analysis
Topic 8 — Interface Analysis — Task Analysis & Modelling
Topic 9 — Interface Analysis — Display Content & Physical Environment
Topic 10 — Interface Design Steps
Topic 11 — Design Issue 1 — System Response Time
Topic 12 — Design Issue 2 — Help Facilities
Topic 13 — Design Issue 3 — Error Handling
Topic 14 — Design Issues 4, 5 & 6 — Menus, Accessibility, Internationalization
Topic 15 — Revised Interface Design Guidelines
Topic 15b — Revised Guidelines for Web Apps
Topic 16 — Interface Design Workflow
Topic 17 — Interface Design Evolution

Topic 1: Introduction to Interface Design

> *“ Technology must now adapt to the human — not the other way around. ”*

Why Did UI Design Evolve?

In the early days of computing, users had to adapt to the machine. Systems were complex, technical, and unfriendly. You had to know commands, syntax, and system internals just to use software. Over time, the philosophy flipped completely — technology must now adapt to the human.

Issues Observed in Bad GUI Designs

Problem	Meaning
Not user-friendly	Hard to figure out how to use
Confusing	User doesn't know what's happening
Counterintuitive	Things don't work the way you'd expect
Frustrating	Errors, dead ends, no guidance

To fix these problems, Theo Mandel defined a set of Golden Rules. The 'window world' interaction mechanism (point, click, drag, menus) was also created as a standard interaction model.

■ Exam Takeaway: UI design shifted from system-centered to human-centered design. Usability is a core requirement of good software engineering.

Topic 2: Golden Rule 1 — Allow Users to Maintain Control

> *❝ The user should always feel in charge of the system — the system should never feel like it is controlling the user. ❞*

6 Sub-Principles

1. Define Interaction Modes with Easy Switching

- The system should have different modes for different tasks
- Users must be able to freely switch between them without getting stuck

✥ *Example: MS Word — editing mode vs spell-check mode; browser — reading mode vs normal mode*

2. Allow Flexible Interaction

- Don't force the user to use only ONE way to interact
- Support multiple input methods — mouse, keyboard, touch, voice, stylus, etc.

✥ *Example: Bold text using toolbar button, or Ctrl+B, or right-click → format*

3. Allow Interaction to be Uninterruptable & Undoable

- User should be able to stop a task mid-way without losing their work
- Undo/Redo must always be available

✥ *Example: Filling a long form and leaving to check something — progress shouldn't be lost*

4. Allow Creating Customized Operations (Macros)

- Advanced users should be able to create shortcuts for frequently repeated tasks
- Macros = a recorded sequence of actions triggered by one command
- Casual users don't need to know this exists — it's hidden from them

✥ *Example: MS Word — record a macro to auto-format a document with one click*

5. Hide Technical Internals from Casual Users

- Regular users should not need to deal with the OS, file systems, or backend complexity
- The interface acts as a shield between the user and system complexity

✥ *Example: When using an app, you never need to type OS-level commands*

6. Design for Direct Interaction with On-Screen Objects

- Objects on screen should feel physical and manipulable
- Users should be able to drag, resize, scale objects as if they were real

✥ *Example: Dragging a file into a folder, or resizing an image by pulling its corner*

■ Exam Takeaway: The user must always feel free, flexible, and in control. The interface should give power to the user — not take it away.

Topic 3: Golden Rule 2 — Reduce User's Memory Load

> *❝ A well-designed system should never make the user memorize things — the interface itself should do the remembering. ❞*

5 Sub-Principles

1. Reduce the Demand to Learn & Recall

- Users should not have to remember past inputs or previous states
- The system saves and visually shows previous choices
- Use list boxes, checkboxes, dropdowns instead of blank text fields wherever possible

✦ *Example: Dropdown showing recent/saved cities instead of typing the city name every time*

2. Establish Meaningful Defaults

- The system should come with sensible pre-set values
- Users can customize, but a Reset to Default option brings everything back

✦ *Example: Printer dialog defaults to 1 copy, A4 size, color*

3. Define Intuitive Shortcuts

- Shortcuts should be easy to guess or remember based on logic
- Bad shortcuts (random key bindings) force memorization

✦ *Example: Ctrl+P for Print, Ctrl+S for Save, Ctrl+Z for Undo — the letter relates to the action*

4. Visual Layout Should Mirror Real-World Processes

- The steps in the software should match how the task works in real life
- Users bring real-world expectations — don't violate them

✦ *Example: Online bill payment should follow same sequence as paying in real life*

5. Disclose Information Progressively

- Don't dump all information at once — show it in layers
- Start simple, reveal detail only when the user asks for it

✦ *Example: E-commerce: product photo on listing → click for details → click for specs → click for reviews*

■ Exam Takeaway: The interface should act as the user's external memory. Always show, don't ask users to recall.

Topic 4: Golden Rule 3 — Make a Consistent Interface

> *❝ The interface should feel familiar — whether it's your first screen or your hundredth, things should work the same way. ❞*

3 Sub-Principles

1. Allow Users to Put Tasks in Meaningful Context

- The user should always know where they are and what they're doing in the system
- The interface must provide contextual clues at all times

❖ *Example: Windows should always have clear titles; consistent color coding — red = error, green = success*

2. Maintain Consistency Across a Family of Applications

- If a company makes multiple applications, they should all follow the same design pattern
- Users learn one app and can immediately feel comfortable in the others

❖ *Example: MS Office — Word, Excel, PowerPoint all share the same ribbon layout, File menu, and shortcuts*

3. Don't Change Things Against the User's Past Habits

- Users carry expectations from previous experience — don't break them without strong reason
- Redefining familiar actions causes confusion and frustration
- Only break conventions if you absolutely must — and even then, warn the user

❖ *Example: Ctrl+S has meant Save for decades. Binding it to Scale instead will cause users to lose work*

Summary of All 3 Golden Rules

Rule	Core Idea	Key Word
Rule 1	User stays in charge	Control
Rule 2	System does the remembering	Memory
Rule 3	Everything feels familiar & predictable	Consistency

■ Exam Takeaway: Consistency means no surprises. Same layout, colors, shortcuts, and flow — across screens, applications, and past experience.

Topic 5: UI Design Models

> *“ An effective design is one in which the user mental model and the implementation model coincide. ”*

The 4 UI Design Models

1. User Model

- The profile of the system's users — background, skill level, goals, behavior
- Collected by the designer to understand the user

2. Design Model

- The designer's representation of the system
- Created based on requirements — incorporates data, architectural, interface and procedural design

3. Implementation Model

- The actual built system — what the user actually sees and interacts with
- Includes the interface look, feel, and behavior

4. Mental Model (System Perception)

- The image the user builds in their head about how the system works
- Formed by using the interface — icons, labels, layout, behavior
- The interface is the only window the user has into the system

> *“ Effective design = the user's mental model matches the implementation model. If there is a gap between these two → confusion, errors, frustration. ”*

■ Exam Takeaway: There are 4 models — user, design, implementation, and mental. A great UI is achieved when what the user expects = what the system actually does.

Topic 6: UI Analysis & Design Process

> *“ UI design uses the spiral process model — you go through the steps again and again, improving each time. ”*

The 4 Steps of the Process

1. Interface Analysis

- Understand before you design
- Study the users, their tasks, the environment, and the content to be displayed
- Output: An analysis model for the interface

2. Interface Design

- Define the objects, actions, and layout of the interface
- Create sketches and storyboards of how screens will look
- Apply design patterns and handle design issues

3. Interface Implementation / Construction

- Actually build the interface using a prototyping approach
- Start with rough prototypes → refine based on feedback

4. Interface Validation

- Test the interface against requirements
- Check: Is it usable? Does it match user expectations?
- Feedback from validation feeds back into analysis → design → construction again

Why Spiral / Iterative?

Because UI design is never perfect on the first try. Each loop through the 4 steps reveals new problems, incorporates user feedback, and improves the design progressively.

■ Exam Takeaway: UI design follows a 4-step iterative spiral: Analysis → Design → Construction → Validation. It keeps repeating because good UI evolves through continuous feedback.

Topic 7: Interface Analysis — User Analysis

> *“ Understand the problem before designing the solution — start by understanding your users. ”*

What is User Analysis?

It means gathering information about who will use the system. You need to understand: the users, the system goals, the displayed content, and the environment.

How to Gather User Information — 3 Sources

1. User Interviews

- Development team members directly meet end users
- Understand their needs, work culture, and daily routines

2. Marketing & Sales Input

- Take market surveys
- Identify in how many ways the product could be useful

3. Support Input

- Support staff collects feedback from existing users
- Identifies what works and what doesn't, which features are easy or difficult

User Analysis Questions

Category	Questions Asked
Profession	Are users trained professionals, technicians, clerical, or manufacturing workers?
Education	Level of formal education of an average user?
Learning style	Can they learn from written materials or do they need classroom training?
Tech comfort	Are they expert typists or keyboard-phobic?
Demographics	Age range? Gender?
Work pattern	Normal office hours or until job is done?
Usage frequency	Use software mostly or occasionally?
Language	Primary spoken language?
Error impact	Consequences of a user mistake in the system?
Domain expertise	Are users experts in the subject matter the system addresses?
Tech interest	Are users interested in the technology behind the interface?
Compensation	How are users compensated for their work?

■ Exam Takeaway: User analysis is about knowing your user before designing for them. The answers to these questions directly drive every design decision.

Topic 8: Interface Analysis — Task Analysis & Modelling

> *❝ Task analysis identifies the tasks users will perform to accomplish system goals — and models how they do it. ❞*

4 Components of Task Analysis & Modelling

1. Use Cases

- Define the basic interaction between an actor (user) and the system
- Written using informal paragraphs — simple, non-technical language
- Used to derive tasks, sub-tasks and interface requirements

✥ *Example: A customer logs in, searches for a product, adds it to cart, and checks out*

2. Task Elaboration (Stepwise Refinement)

- Break down high-level tasks into smaller, detailed sub-tasks
- Can be derived manually or from a pre-existing system

✥ *Example: Checkout → enter address → select payment → confirm order → get receipt*

3. Object Elaboration

- Extract physical objects from the system description
- Define their classes and behavior
- These objects become the basis for UI elements (buttons, forms, lists)

✥ *Example: In a shopping system — objects: Product, Cart, Order, Payment*

4. Workflow Analysis

- Defines how a work process is completed when multiple people and roles are involved
- Shows the flow of tasks across different users/roles
- Represented using a UML Swim Lane Diagram

Swim Lane Diagram

A UML diagram where each role/actor gets their own lane. Tasks flow across lanes showing who does what and when — making it easy to see handoffs between roles.

✥ *Example: Order system: Customer places order → Sales Rep confirms → Warehouse packs & ships → Customer receives*

Task Identification Answers These Questions

- What tasks and sub-tasks does the user perform?
- In what sequence are tasks performed? (workflow)
- In what specific circumstances does a task occur?

■ Exam Takeaway: Task analysis breaks down what users do into use cases, sub-tasks, objects, and workflows. Key output: swim lane diagrams showing task flow across roles.

Topic 9: Interface Analysis — Display Content & Physical Environment

> *❝ The final part of Interface Analysis — what to display and where the system will be used. ❞*

Part 1: Analysis of Display Content

1. Format & Aesthetics of Content

- How should information be visually presented?
- What is the right layout, font, color, spacing?
- Aesthetics matter — a well-formatted screen is easier to process

2. Stepwise Refinement Approach

- Content is not dumped all at once — revealed progressively, level by level

✥ *Example: E-commerce: product image first → click for specs → click for reviews*

Part 2: Analysis of Physical Work Environment

1. Environment Must Support Proper Operation & Concentration

- The physical setting should not hinder the user from using the system

✥ *Example: Noisy factory floor → voice input may not work → keyboard/touch better*

2. Environment Must Match Software Aesthetics

- The look and feel of the software should make sense in its physical context

✥ *Example: Dark-themed UI for a dimly lit control room, bright UI for a well-lit office*

Complete Interface Analysis Summary

Analysis	Question Answered
User Analysis	WHO are the users?
Task Analysis	WHAT do they do?
Display Content Analysis	WHAT do they need to see?
Physical Environment Analysis	WHERE will they use it?

■ Exam Takeaway: Display content analysis defines what to show and how. Physical environment analysis ensures the real-world setting is compatible with the designed interface.

Topic 10: Interface Design Steps

> *❝ Using information developed during interface analysis, define the objects, actions, events and states of the interface. ❞*

4 Core Design Steps

1. Define Interface Objects & Operations

- Identify all objects the user will interact with (buttons, text fields, icons, video players)
- Identify all operations/tasks linked to those objects

✦ *Example: A Print button → operation is printing a document*

2. Define Events That Change UI State

- An event is a user action — click, type, drag, hover
- Each event causes the UI to change its state
- Designer must map out all possible events and their resulting states

✦ *Example: Clicking Login → UI transitions from login screen to dashboard*

3. Depict Each Interface State as it Looks to the User

- Create sketches or wireframes of every screen/state
- Helps catch layout and usability issues early — before coding

4. Show How User Infers System State from Interface

- The interface must always communicate its current state clearly
- User should never be confused about what the system is doing

✦ *Example: Loading spinner = system is processing; green checkmark = action was successful*

Object Categories

Object Type	Meaning	Example
Source Object	Produces/initiates something	Printer icon
Target Object	Receives the action	Paper/document
Application Object	Core entity in the system	Screen, database table

UI Design Patterns

A design pattern is a proven solution for a common UI problem. Don't reinvent the wheel — use established patterns.

✦ *Example: For a calendar UI → use a Calendar Strip pattern that shows dates in a horizontal strip, highlights current date, and allows navigation*

■ Exam Takeaway: Interface design steps translate analysis into actual UI by defining objects, events, states and sketches. Objects are categorized as source, target or application objects.

Topic 11: Design Issue 1 — System Response Time

> *❝ System response time is the primary complaint for almost every interactive system. ❞*

What is System Response Time?

It is the time the system takes to respond to a user's action — from when the user does something to when the system reacts.

Two Key Characteristics

1. Length (How Long it Takes)

- Longer response times = user frustration
- Users expect quick feedback — if the system takes too long, they assume something is wrong
- General rule: the shorter the response time, the better the experience

❖ *Example: A page that takes 10 seconds to load feels broken compared to one that loads in 1 second*

2. Variability (How Consistent it Is)

- Varying response times = user confusion
- If sometimes the system responds in 1 second and other times in 8 seconds — users don't know what to expect
- Inconsistency is just as damaging as slowness

❖ *Example: A search that sometimes returns results instantly and sometimes takes 7 seconds — users lose trust*

Summary

Problem	User Reaction
Too long	Frustration, abandonment
Too variable	Confusion, loss of trust

■ Exam Takeaway: System response time has two characteristics — length and variability. Responses must be both fast and consistent.

Topic 12: Design Issue 2 — Help Facilities

> *“ Help facilities must be carefully designed — not just as an afterthought. ”*

Two Types of Help Facilities

1. Online Helpdesk

- Digital, in-app help available while using the system
- Immediate and contextual

2. Detailed User Manuals

- Comprehensive printed or digital documentation
- For deeper reference outside the system

4 Key Design Questions

1. Is Help Available for ALL Functionalities?

- Help should not be selective — every feature needs coverage
- Partial help is almost as bad as no help

2. How Will the User Request Help?

- Method to access help must be intuitive
- A help menu in the navigation, or a function key mapped to help (e.g., F1)
- User should never have to search hard just to find help itself

3. How is Help Presented?

- A separate window that opens alongside the current screen, or a printed manual
- Presentation must not disrupt the user's current work

4. How Does the User Return to Normal UI After Help?

- After consulting help, returning to the main interface must be seamless and easy
- User should not lose their place or their work

✦ *Example: Closing a help window brings you straight back to exactly where you were*

■ Exam Takeaway: Help facilities: online helpdesk and user manuals. Key questions: Is help available for everything? How requested? How presented? How to return after?

Topic 13: Design Issue 3 — Error Handling

> *❝ Error messages and warnings are bad news to users — so they must be delivered in the best possible way. ❞*

4 Qualities of a Good Error Message

1. Describe the Problem Simply

- Use plain, non-technical language
- The user should immediately understand what went wrong

✦ *Example: Bad: 'Exception 0x80070057 parameter incorrect' | Good: 'The file could not be saved because the disk is full'*

2. Provide Valuable Advice

- Don't just report the error — tell the user what to do about it
- Include hyperlinks to remedies or next steps where possible

✦ *Example: 'Your password is incorrect. Click here to reset it.'*

3. List the Negative Consequences of the Error

- Tell the user what impact the error has had
- Did they lose data? Is the process stopped? Does it affect other parts?

✦ *Example: 'Your session has expired. Any unsaved changes have been lost.'*

4. Use Visual Prompts with a Clear Color Scheme

- The message must be visually distinct — user should immediately recognize it as an error

Color	Meaning
Red	Error — something went wrong
Yellow/Orange	Warning — potential problem
Green	Success — action completed
Blue	Information — neutral message

■ Exam Takeaway: A good error message: describes the problem simply, provides advice, lists consequences, and uses clear visual color coding.

Topic 14: Design Issues 4, 5 & 6 — Menus, Accessibility, Internationalization

Design Issue 4 — Menu & Command Labelling

> *❝ How commands are assigned to menu items so the system can run with or without the UI. ❞*

Two Styles of Interaction Architecture

- Command Oriented Architecture — user types text commands (e.g., terminal/command line)
- UI Based Architecture — user interacts through graphical menus and buttons (e.g., MS Office)

Both styles are used frequently. Every menu item should have a keyboard command equivalent to allow power users to operate the system without touching the UI.

✦ *Example: File → Save has the command equivalent Ctrl+S*

Design Issue 5 — Application Accessibility

> *❝ UI must cater to the needs of physically challenged persons as well. ❞*

The interface must be usable by everyone — including people with disabilities. Cannot design only for able-bodied users.

Examples of Accessibility Features

- Voice input for users who cannot use keyboard/mouse
- Screen readers for visually impaired users
- High contrast modes for users with visual difficulties
- Keyboard-only navigation for users who cannot use a mouse

Design Issue 6 — Internationalization

> *❝ UI must target the global market and follow global design standards. ❞*

Two Requirements

- Follow global design standards — patterns universally understood across cultures
- Facilitate multiple languages — UI must support switching between languages easily
- Must also consider text direction (e.g., Arabic reads right to left)

✦ *Example: A floppy disk icon for save, a magnifying glass for search — recognized globally*

All 6 Design Issues — Complete Summary

Issue	Topic	Key Concern
1	System Response Time	Length & variability
2	Help Facilities	Availability, access, presentation, return
3	Error Handling	Simple, advisory, consequential, visual
4	Menu & Command Labelling	UI + command equivalents
5	Accessibility	Cater to physically challenged users
6	Internationalization	Global standards & multiple languages

■ **Exam Takeaway: These 3 issues ensure the UI is operable (menus/commands), inclusive (accessibility), and global (internationalization).**

Topic 15: Revised Interface Design Guidelines

> *❝ In interactive systems, completing tasks quickly matters — so relevant options must be easily accessible and appropriately sized. ❞*

8 Revised Guidelines

1. Anticipation

- The application must predict the user's next move
- Design should guide users to their next step automatically
- Users shouldn't have to search for what comes next

✦ *Example: Software installation steps defined to predict and lead the user to the next step*

2. Communication

- The application must show the status of any activity initiated by the user
- User should always know what the system is doing

✦ *Example: File copying shown via a progress bar*

3. Consistency

- Navigation controls, menus, icons and aesthetics must be consistent throughout the entire application
- Same elements should always mean the same thing

✦ *Example: A yellow triangle always means a warning — never use it for anything else*

4. Controlled Autonomy

- A revision of Golden Rule 1 — give user control, but restrict controls based on user's role
- Not every user should have access to everything

✦ *Example: Enforce ID and password — a regular user cannot access admin options*

5. Efficiency

- Design must work towards the user's efficiency
- Reduce unnecessary steps or formatting requirements

✦ *Example: Inputting CNIC or phone number without spaces or hyphens reduces processing time*

6. Focus

- The interface should help the user stay focused
- Show relevant options and data first — don't distract with irrelevant content

7. Fitt's Law

> *❝ The time to acquire a target is a function of the distance to the target and the size of the target. ❞*

- Bigger buttons that are closer = faster to reach and click
- This is the scientific basis behind making important buttons larger and grouping related items together

8. Latency Reduction

- The interface must do multitasking rather than keeping the user waiting
- Keep the user engaged while processing happens in the background

✦ *Example: Display a progress bar while downloading, or show animated visuals to keep user focused*

Topic 15b: Revised Interface Design Guidelines for a Web App

5 Web-Specific Guidelines

1. Learnability

- Minimize learning time by creating simple and intuitive designs
- A new user should be able to figure out the web app quickly without training

2. Maintain Work Product Integrity

- The interface should autosave user data
- Losing data due to an error is extremely frustrating

✦ *Example: Filling a long form and losing all data because of one incorrect field — correct fields should be autosaved*

3. Readability

- All information presented must be readable by all age group users
- Font size, contrast, and layout must be accessible to everyone

4. Track State

- Use cookies to track user activities
- User can log out and when they log back in, they continue from the same state

✦ *Example: Items still in your cart after logging back into a shopping site*

5. Visible Navigation

- Obvious navigation bars must be present on every interface of the application
- User should never feel lost or unable to find where to go next

■ **Exam Takeaway: Revised guidelines: 8 general principles (Anticipation, Communication, Consistency, Controlled Autonomy, Efficiency, Focus, Fitt's Law, Latency Reduction) + 5 web-specific ones (Learnability, Work Integrity, Readability, Track State, Visible Navigation).**

Topic 16: Interface Design Workflow

> *❝ A structured sequence of steps that takes you from raw requirements to a refined, validated interface model. ❞*

The 9 Steps of Interface Design Workflow

1. Derive and Refine Information from Requirements

- Start by extracting all relevant information from the system requirements
- Understand what the system must do before designing how it looks

2. Develop Rough Layout Sketch

- Create a basic, rough drawing of the interface
- No need for detail yet — just layout and structure, like a blueprint before building

3. Map User Objectives to Actions

- Identify what the user wants to achieve and map it to specific interface actions

✦ *Example: Hotel booking app: User objective = Book a room → Actions: Home, Search, Booking, Facilities, Contact*

4. Create Storyboard Screen Images for Every Interface

- Draw detailed screen-by-screen visuals of the entire interface
- Shows exactly what the user will see at each step — like a comic strip of the user's journey

5. Refine Layouts by Improving Aesthetics

- Go back to the rough sketches and polish them
- Improve colors, spacing, typography, alignment

6. Develop an Activity Diagram to Show Design Flow

- Create a UML Activity Diagram showing the flow of actions
- Shows how the user moves through the system — captures decision points and parallel actions

7. Develop State Diagrams to Show Changes in State

- Create UML State Diagrams showing how the UI changes
- Every user action that causes a state transition is captured here

✦ *Example: Login screen → Dashboard → Settings → Logout → Login screen*

8. Describe Interface Layout for Every State

- For each state, document the layout
- What does the screen look like? What elements are visible, hidden, enabled, or disabled?

9. Refine & Review the Model

- Go back through everything and review and refine
- Check for consistency, usability, and completeness
- Get feedback and improve before moving to implementation

■ Exam Takeaway: The interface design workflow is a 9-step process: requirements → rough sketches → storyboards → activity & state diagrams → final refined model.

Topic 17: Interface Design Evolution

> *“ Interface design has evolved from users adapting to machines — to machines adapting to users. ”*

The Evolution Journey

1. Early Systems — Command Line Era

- No graphical interface at all
- Users had to type exact commands to operate the system
- Required technical knowledge and memorization
- Only trained professionals could use computers

✦ *Example: MS-DOS — commands like cd, dir, copy*

2. The Window World — GUI Era

- Introduction of Graphical User Interfaces (GUI)
- The window world was created — windows, icons, menus, pointers
- Computers became accessible to non-technical users
- Interaction shifted to point, click, drag

✦ *Example: Early Windows OS, Apple Macintosh*

3. Golden Rules & Design Standards Era

- Designers recognized patterns of good and bad UI
- Theo Mandel and others defined golden rules and guidelines
- UI design became a discipline — not just an afterthought
- Consistency, usability and user control became priorities

4. Modern Era — User Centered Design

- Technology now fully adapts to the human
- Multiple input methods — touch, voice, gesture, pen
- Responsive design — UI adapts to different devices and screen sizes
- Accessibility — UI designed for everyone including differently abled users
- Internationalization — UI targets global markets

✦ *Example: Smartphones, tablets, smart TVs, wearables*

The Full Evolution at a Glance

Era	Key Feature	User Type
Command Line	Type commands	Technical experts only
GUI / Window World	Point & click	General public
Design Standards	Golden rules, consistency	Broader audience
Modern / UCD	Touch, voice, responsive	Everyone globally

■ Exam Takeaway: Interface design evolved from complex command-based systems requiring technical expertise, to modern user-centered interfaces that adapt to any user, device, language, and ability.

Quick Revision Summary

Topic	Title	Key Point
1	Introduction	UI shifted from system-centered to human-centered
2	Golden Rule 1	User control — flexible, undoable, customizable
3	Golden Rule 2	Reduce memory load — defaults, shortcuts, progressive disclosure
4	Golden Rule 3	Consistency — context, family apps, past habits
5	UI Design Models	4 models; mental model must match implementation model
6	UI Process	4-step spiral: Analysis → Design → Construction → Validation
7	User Analysis	Interviews, marketing, support; 12 user categorization questions
8	Task Analysis	Use cases, task elaboration, object elaboration, workflow/swim lanes
9	Display & Environment	Progressive disclosure; environment must match software
10	Design Steps	Objects (source/target/app), events, states, patterns
11	Response Time	Length + variability both matter
12	Help Facilities	Online & manual; 4 design questions
13	Error Handling	Simple, advisory, consequential, visual color scheme
14	Issues 4,5,6	Menu commands, accessibility, internationalization
15	Revised Guidelines	8 guidelines: Anticipation to Latency Reduction
15b	Web App Guidelines	5 guidelines: Learnability to Visible Navigation
16	Design Workflow	9 steps from requirements to final refined model
17	Evolution	CLI → GUI → Standards → Modern UCD